Instructor Guide: JavaScript, DOM, JSON and APIs

One-day workshop for 1st-year students. Written for the final Interactive_JavaScript_Workshop deck (19 slides); slide numbers below are that deck's. Every snippet can be pasted into the Console or a file and run.

Workshop at a glance

Students leave with a deployed **Student Task Manager**: add, complete, delete and filter tasks, saved across refreshes, with a Daily Tip banner fetched from an API. It is built from three linked files (`index.html` , `style.css` , `script.js`) and matches the mock-up on slide 14. The deck's roadmap (slide 2) sets the three sessions; each topic below has ten short examples you can paste into the Console or a file.

Time	Slides	Topic	Hands-on
9:00–9:15	1–2	Roadmap, workspace setup, DevTools hook	Edit a live page in DevTools
9:15–9:25	3	Preview: what happens when I click Add?	Trace the chain on the slide
9:25–9:45	4	Variables and scope (Topic 1)	Predict outputs
9:45–10:05	5	Equality, conditions, the Truth Machine (Topic 2)	Code drill
10:05–10:30	6	Arrays and objects (Topic 3)	`map` , `filter` , `find` on a student list
10:30–11:00	7	The DOM (Topic 4)	Change a page with JS
11:00–11:30		Break	
11:30–12:00	8	Events and validation (Topic 5)	Registration form

Time	Slides	Topic	Hands-on
12:00–12:10	9	Safe rendering, the `innerHTML` nuance	XSS demo
12:10–12:40	10	localStorage and JSON (Topic 6)	Student table that survives refresh
12:40–1:30	11–13	APIs, fetch, async/await (Topic 7)	Fetch and render JSON
1:30–2:30		Lunch	
2:30–2:40	17–18	Viva warm-up and the frontend loop	Team quiz
2:40–2:50	14	Blueprint of the Task Manager	Sketch the data model
2:50–3:05	15–16	Git branching and the lifecycle (Topic 8)	Clone, branch
3:05–3:50		Build sprints in pairs	Task Manager
3:50–4:00	19	Merge, deploy to GitHub Pages	Live URL

Before the day

Git and the network are the riskiest parts, so prepare those first.

- **Accounts, a week ahead.** Every student needs a GitHub account, Git installed, `git config --global user.name` and `user.email` set, and a successful test `git push` from the campus network (VS Code's GitHub sign-in is the easiest route).
- **Repositories.** Upload the `student-task-manager-starter` folder (from the zip) as a GitHub *template repository*, so each pair clicks "Use this template". Keep `student-task-manager` (the finished app) private to you until the end. Use public repositories so GitHub Pages works on free accounts.
- **Pairs.** Student A owns the repo and adds Student B under Settings, then Collaborators.
- **Check the API.** Open `https://dummyjson.com/quotes/random` from the campus network. If it is blocked the app still works, because it shows a built-in tip.
- **Tools.** VS Code with the Live Server extension, Chrome or Edge, DevTools open (F12). Keep StackBlitz as a backup for laptops that fail.

- **Git fallback.** If GitHub or wifi fails, pairs use local branches and `git merge`; skip push and pull requests.
- **Helpers.** Two or three senior students as roaming helpers, with a sticky note on the lid as the “stuck” signal.
- **Your own copy.** Run the finished app once before the day and keep it open in a tab for the demo.

Deck notes

The deck is solid. These are the spots to handle aloud or adjust.

Slide	Note	What to do
3	The code in the callout is garbled, and it shows a table where the app uses a list	Say the real line: `button.addEventListener('click', () => { ... })`. Table or list, the chain is identical
4–6	The deck has no slide for functions, loops or `if/else`	Topic 2’s examples cover them
7	Shows `getElementById`, `textContent`, `style`, `createElement`	Add `querySelectorAll` and `classList` from Topic 4
12	The JSON has a nested `contact` object	Use it to teach `data[0].contact.city`
13	The URL is the placeholder `api.example.com`	Live-code with `https://dummyjson.com/quotes/random`
14	Mock-up has a tip banner, filters and delete icons	The repo matches it exactly
16	Shows `git init`	Pairs start from a template and use `git clone`; `git init` is for brand-new local projects
19	Menu names on GitHub change from time to time	Do the steps live and adapt

The deck has no slide for CSS or Flexbox. The finished `style.css` is complete in the repository, so students never write CSS.

Session 1: Core JavaScript and the DOM (9:00–11:00)

Run every example in the Console (F12) one block at a time. Before each block, ask “what will this print?”, take a show of hands, then run it. Reload the page between topics so names don’t clash.

Topic 1: Variables, scope and const (slide 4, 20 min)

Teach: `let` changes, `const` is a fixed reference, `var` is legacy. The slide’s warning matters most: `const` stops reassignment, not mutation.

```
// 1. let can be reassigned
let score = 70;
score = 75;
console.log(score);              // 75

// 2. const cannot
const college = 'RVCE';
// college = 'BMS';              // TypeError: Assignment to constant
variable.

// 3. const object: the contents can still change (slide 4’s warning)
const student = { name: 'Anita', marks: 80 };
student.marks = 85;              // allowed
console.log(student);            // { name: 'Anita', marks: 85 }
// student = {};                 // TypeError

// 4. const array: same rule
const subjects = ['Maths', 'Physics'];
subjects.push('Chemistry');      // allowed
console.log(subjects.length);    // 3

// 5. let is block-scoped
if (true) { let blockLet = 'inside'; }
// console.log(blockLet);        // ReferenceError: blockLet is not defined

// 6. var leaks out of blocks
if (true) { var leaky = 'I escaped'; }
console.log(leaky);              // 'I escaped'

// 7. var can be redeclared by accident; let cannot
var x = 1;
var x = 2;                       // silently allowed
```

```
// let y = 1; let y = 2;          // SyntaxError: already declared

// 8. Loop scope: let gives each pass its own i
for (let i = 0; i < 3; i++) setTimeout(() => console.log(i), 0);   // 0 1 2
for (var j = 0; j < 3; j++) setTimeout(() => console.log(j), 0);   // 3 3 3

// 9. Template literals (backticks and ${ })
const studentName = 'Anita', marks = 85;
console.log(`${studentName} scored ${marks} marks`);

// 10. typeof
console.log(typeof 'text', typeof 42, typeof true, typeof undefined, typeof
null);
// string number boolean undefined object  (null printing 'object' is an old
quirk)
```

Watch for: students assume `const` means frozen. Example 3 fixes that. Rule to teach: default to `const`, use `let` when the value changes, never `var`.

Topic 2: Equality, conditions and the Truth Machine (slide 5, 20 min)

Teach: always use `===`. Start with the slide's drill (`student.marks` prints 85; `status` throws a ReferenceError because it was block-scoped). The deck has no slide for loops or functions, so examples 6 to 10 cover them.

```
// 1. Loose equality converts types
console.log('5' == 5);           // true

// 2. Strict equality checks the type too
console.log('5' === 5);          // false

// 3. More coercion surprises
console.log(0 == false, 0 === false);           // true false
console.log(null == undefined, null === undefined);   // true false

// 4. + glues text, - converts to numbers
console.log('5' + 3, '5' - 3);    // '53' 2

// 5. Input is always text: convert before maths
const raw = '85';
const marks = Number(raw);
console.log(marks + 5);          // 90
```

```
// 6. if / else if / else
if (marks >= 90) console.log('A+');
else if (marks >= 75) console.log('A');
else console.log('B or below');   // A

// 7. Ternary
console.log(marks >= 40 ? 'Pass' : 'Fail');   // Pass

// 8. Logical operators
const hasId = true, feesPaid = false;
console.log(hasId && feesPaid, hasId || feesPaid, !feesPaid);   // false true
true

// 9. switch
const branch = 'CSE';
switch (branch) {
  case 'CSE': console.log('Computer Science'); break;
  case 'ISE': console.log('Information Science'); break;
  default: console.log('Other');
}

// 10. A function and a loop together
function grade(p) {
  if (p >= 90) return 'A+';
  if (p >= 75) return 'A';
  if (p >= 40) return 'C';
  return 'F';
}
for (let m = 30; m <= 90; m += 30) console.log(m, grade(m));   // 30 F, 60 C,
90 A+
```

Activity: write `isEven(n)` and print the even numbers from 1 to 20.

Topic 3: Arrays and objects (slide 6, 25 min)

Teach: an array is an ordered list, an object is key and value pairs, and real data is an array of objects. `map`, `filter`, `find` and `splice` are the four tools the Task Manager uses.

```
// 1. Index and length
const marks = [70, 85, 90];
console.log(marks[0], marks.length);     // 70 3

// 2. push and pop work on the end
```

```
marks.push(95);
marks.pop();                              // back to [70, 85, 90]

// 3. unshift and shift work on the start
marks.unshift(60);
marks.shift();

// 4. splice inserts or removes anywhere
marks.splice(1, 0, 80);                   // insert 80 at index 1: [70, 80,
85, 90]
marks.splice(2, 1);                       // remove 1 item at index 2: [70,
80, 90]

// 5. map transforms every item into a NEW array
console.log(marks.map(m => m + 5));       // [75, 85, 95]

// 6. filter keeps matching items in a NEW array
console.log(marks.filter(m => m >= 75)); // [80, 90]

// 7. find and findIndex locate one item
const students = [
  { id: 1, name: 'Rahul', marks: 80, status: 'Active' },
  { id: 2, name: 'Anita', marks: 92, status: 'Active' }
];
console.log(students.find(s => s.id === 2));        // Anita’s object
console.log(students.findIndex(s => s.id === 2));   // 1

// 8. forEach and reduce
students.forEach(s => console.log(s.name));
console.log(students.reduce((sum, s) => sum + s.marks, 0));   // 172

// 9. Object access and update
const rahul = students[0];
console.log(rahul.name, rahul['marks']);   // dot and bracket access
rahul.marks = 85;
rahul.city = 'Mysuru';                     // add a new key

// 10. Nested objects (slide 12), destructuring, spread
const record = { id: 1, name: 'Anita', contact: { email: 'anita@example.com',
city: 'Bengaluru' } };
const { name: studentName, contact } = record;
console.log(studentName, contact.city);    // Anita Bengaluru
const updated = { ...record, name: 'Anita K' };   // copy with one change
```

Activity: from `students`, print the names of everyone with marks of at least 85. **Watch for:** `map` and `filter` return new arrays and never change the original.

Topic 4: The DOM and safe rendering (slides 7 and 9, 30 min)

Teach: the DOM is the browser's tree of your HTML. Four moves: select, change, style, create. Then slide 9's rule: clearing with `innerHTML = ''` is safe, putting user or fetched text into `innerHTML` is not.

```
<h1 id="title">Hello</h1>
<p class="note">First note</p>
<p class="note">Second note</p>
<ul id="list"></ul>
<button id="go">Go</button>
```

```
// 1. Select one element
const title = document.getElementById('title');

// 2. querySelector uses CSS selectors; querySelectorAll returns all matches
const firstNote = document.querySelector('.note');
console.log(document.querySelectorAll('.note').length);   // 2

// 3. Change text
title.textContent = 'Hello Students';

// 4. Change style
title.style.color = 'blue';

// 5. Classes are cleaner than inline styles
title.classList.add('highlight');
title.classList.toggle('highlight');      // adds if missing, removes if
present

// 6. Attributes
const go = document.getElementById('go');
go.setAttribute('disabled', true);
go.removeAttribute('disabled');

// 7. Create and append
const list = document.getElementById('list');
const li = document.createElement('li');
li.textContent = 'Task 1';
```

```
list.appendChild(li);

// 8. Render from data: loop, create, append
['HTML', 'CSS', 'JS'].forEach(skill => {
  const item = document.createElement('li');
  item.textContent = skill;
  list.appendChild(item);
});

// 9. Remove an element
li.remove();

// 10. Safe versus unsafe (slide 9)
const evil = '<img src=x onerror="alert(1)">';
list.innerHTML = '';              // SAFE: only clearing
list.innerHTML = evil;            // UNSAFE: the browser runs the attacker's
code
list.textContent = evil;          // SAFE: shown as plain text
```

Activity (about 10 min): three buttons that change a heading, a paragraph and the page background. **Watch for:** `getElementById('title')` is case-sensitive and returns `null` if the id is wrong, which causes the classic "Cannot read properties of null".

Session 2: Events, storage and APIs (11:30–1:30)

Start with a 5-minute recap quiz from Session 1. Students now make pages react, remember and fetch.

Topic 5: Events and validation (slides 3 and 8, 30 min)

Teach: walk slide 3's chain once more (click, listener, read input, validate, create object, push, render). Then slide 8: `addEventListener`, `preventDefault`, `trim`, regex. Show regex but don't teach it.

```
<form id="regForm">
  <input id="name" placeholder="Name"> <span id="nameError" class="error">
</span>
  <input id="mobile" placeholder="Mobile"> <span id="mobileError"
class="error"></span>
  <input id="pass" type="password" placeholder="Password">
  <input id="pass2" type="password" placeholder="Repeat password">
  <button type="submit">Register</button>
</form>
<button id="count">Clicked 0 times</button>
```

```
const form = document.getElementById('regForm');
const nameBox = document.getElementById('name');
const mobile = document.getElementById('mobile');

// 1. click event
const btn = document.getElementById('count');
let clicks = 0;
btn.addEventListener('click', () => {
  clicks++;
  btn.textContent = `Clicked ${clicks} times`;
});

// 2. the event object: event.target is the element that fired it
btn.addEventListener('click', event => console.log(event.target.id));   //
count

// 3. input event fires on every keystroke; event.target.value is the current
text
nameBox.addEventListener('input', event => console.log(event.target.value));

// 4. submit + preventDefault stops the page reload
form.addEventListener('submit', event => {
  event.preventDefault();
  console.log('Handled by JavaScript');
});

// 5. trim removes accidental spaces
console.log('  Anita '.trim());                // 'Anita'

// 6. length check
const pin = '123456';
```

```
console.log(pin.length === 6);                  // true

// 7. regex: exactly 10 digits (slide 8)
const mobilePattern = /^\d{10}$/;
console.log(mobilePattern.test('9876543210'));  // true
console.log(mobilePattern.test('98765'));       // false

// 8. a simple email check
const email = 'anita@example.com';
console.log(email.includes('@') && email.includes('.'));   // true

// 9. password match
const pass = document.getElementById('pass').value;
const pass2 = document.getElementById('pass2').value;
console.log(pass === pass2);

// 10. one validator returns an error message or an empty string; the form
shows it
function validateMobile(value) {
  if (value.trim() === '') return 'Mobile is required';
  if (!/^\d{10}$/.test(value.trim())) return 'Enter exactly 10 digits';
  return '';
}
form.addEventListener('submit', event => {
  event.preventDefault();
  document.getElementById('mobileError').textContent =
validateMobile(mobile.value);
});
```

Activity: make the registration form validate name, mobile and password match without reloading, with errors in a red `<span class="error">`. **Watch for:** forgetting `event.preventDefault()` (the page flashes and the data disappears) and comparing input text with numbers.

Topic 6: localStorage and JSON (slides 10 and 12, 30 min)

Teach: variables live in RAM and vanish on refresh. localStorage keeps text in the browser. JSON is the translator: `stringify` packs, `parse` unpacks. Show DevTools, then Application, then Local Storage.

```
// 1. The four localStorage methods
localStorage.setItem('city', 'Bengaluru');
console.log(localStorage.getItem('city'));   // 'Bengaluru'
localStorage.removeItem('city');
localStorage.clear();                        // removes everything for this
site

// 2. It stores strings only
localStorage.setItem('student', { name: 'Anita' });
console.log(localStorage.getItem('student'));   // '[object Object]' (data
lost)

// 3. stringify packs an object into text
const student = { name: 'Anita', branch: 'CSE' };
const text = JSON.stringify(student);
console.log(text);                           //
'{"name":"Anita","branch":"CSE"}'
localStorage.setItem('student', text);

// 4. parse unpacks text into an object
const back = JSON.parse(localStorage.getItem('student'));
console.log(back.name);                      // 'Anita'

// 5. Stored numbers come back as text
const visits = Number(localStorage.getItem('visits')) || 0;
localStorage.setItem('visits', visits + 1);  // refresh the page and run it
again

// 6. An array of objects (slide 10)
const students = [{ id: 101, name: 'Alice' }, { id: 102, name: 'Bob' }];
localStorage.setItem('students', JSON.stringify(students));
console.log(JSON.parse(localStorage.getItem('students')).length);   // 2

// 7. First visit: nothing is stored, getItem returns null
console.log(localStorage.getItem('nothing'));                    // null
const saved = JSON.parse(localStorage.getItem('nothing')) || [];  // [] on
first visit

// 8. Corrupted data: wrap parse in try/catch
localStorage.setItem('broken', '{not json');
try {
  JSON.parse(localStorage.getItem('broken'));
} catch (error) {
```

```
  console.log('Bad data, starting fresh:', error.name);   // SyntaxError
}

// 9. JSON rules: double quotes only, no trailing commas
// JSON.parse("{name:'Anita'}");        // SyntaxError
// JSON.parse('{"name":"Anita",}');     // SyntaxError
console.log(JSON.parse('{"age":18}').age);   // 18

// 10. Nested JSON shaped like slide 12
const apiText = '[{"id":1,"name":"Anita","usn":"1RV23CS001","contact":
{"city":"Bengaluru"}}]';
const people = JSON.parse(apiText);
console.log(people[0].contact.city);          // 'Bengaluru'
console.log(JSON.stringify(people, null, 2)); // pretty-printed
```

Activity: make the Session 1 student list survive a refresh. **Watch for:** saving without `JSON.stringify`, and forgetting `|| []` so the first visit crashes.

Topic 7: APIs, fetch and async/await (slides 11–13, 50 min)

Teach: slide 11's picture. JavaScript does not wait; `fetch` returns a Promise, an IOU for data that will arrive. Open `https://dummyjson.com/quotes/random` in a tab first so students see raw JSON. Slide 13 uses the placeholder `api.example.com`; use this real URL instead.

```
// 1. Async does not block: this prints 1, 2, 3 (not 1, 3, 2)
console.log('1');
fetch('https://dummyjson.com/quotes/random').then(() => console.log('3'));
console.log('2');

// 2. fetch returns a Promise, not the data
console.log(fetch('https://dummyjson.com/quotes/random'));   // Promise {
<pending> }

// 3. The .then chain (slide 13, left)
fetch('https://dummyjson.com/quotes/random')
  .then(response => response.json())
  .then(data => console.log(data.quote, '-', data.author));

// 4. Always check response.ok (slide 13's rule)
fetch('https://dummyjson.com/quotes/random')
  .then(response => {
```

```
    if (!response.ok) throw new Error('Request failed: ' + response.status);
    return response.json();
  })
  .then(data => console.log(data));

// 5. .catch handles failures
fetch('https://dummyjson.com/quotes/random')
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error('Could not load:', error.message));

// 6. async/await (slide 13, right) reads top to bottom
async function loadQuote() {
  const response = await fetch('https://dummyjson.com/quotes/random');
  const data = await response.json();
  console.log(data.quote);
}
loadQuote();

// 7. try/catch with a 404. Break it on purpose: /quotes/9999 gives a 404
async function loadQuoteSafely(id) {
  try {
    const response = await fetch('https://dummyjson.com/quotes/' + id);
    if (!response.ok) throw new Error('Request failed: ' + response.status);
    console.log((await response.json()).quote);
  } catch (error) {
    console.log('Error:', error.message);
  }
}
loadQuoteSafely(9999);    // Error: Request failed: 404
// Then DevTools, Network, Offline: fetch itself rejects and catch runs

// 8. Render fetched text safely
async function showTip() {
  const response = await fetch('https://dummyjson.com/quotes/random');
  const data = await response.json();
  document.getElementById('tip').textContent = `${data.quote} -
${data.author}`;   // textContent
}

// 9. Loading state and fallback
async function showTipWithFallback() {
  const box = document.getElementById('tip');
```

```
  box.textContent = 'Loading...';
  try {
    const response = await fetch('https://dummyjson.com/quotes/random');
    if (!response.ok) throw new Error(response.status);
    box.textContent = (await response.json()).quote;
  } catch (error) {
    box.textContent = 'Write code for humans, not just machines.';
  }
}

// 10. Query parameters and a list of results
async function showQuotes() {
  const response = await fetch('https://dummyjson.com/quotes?limit=3');
  const data = await response.json();            // { quotes: [...], total,
skip, limit }
  const list = document.getElementById('list');
  list.innerHTML = '';
  data.quotes.forEach(q => {
    const li = document.createElement('li');
    li.textContent = q.quote;
    list.appendChild(li);
  });
}
```

Activity: show three quotes in a list, with a “could not load” message when the request fails. **Watch for:** `fetch` does not reject on a 404, so `response.ok` is the check; and pages opened from `file://` can behave differently, so use Live Server.

Session 3: Task Manager, Git and deployment (2:30–4:00)

Ninety minutes is tight, so pairs start from the starter template: `index.html` and `style.css` are finished, and `script.js` holds numbered TODOs. Students spend the time on JavaScript and Git, not on setup.

Warm-up: viva and the frontend loop (slides 17–18, 10 min)

Split the room into two teams and run slide 17’s four questions as a buzzer quiz; the answers are on the slide. Then slide 18, the loop that is also the app’s architecture. Map each step to a function in the project:

Slide 18 step	In the Task Manager
1. User action	Click Add, tick a box, click a filter
2. Event capture	`addEventListener` calls
3. State change	The `tasks` array
4. Persistence and network	`saveTasks()`, `loadDailyTip()`
5. DOM update	`renderTasks()`

Blueprint (slide 14, 10 min)

Show your finished app next to the slide's mock-up: tip banner, input and Add button, All/Pending/Completed filters, checkbox rows with a delete icon, grey line-through for completed tasks. The data model is `{ id, title, completed }`, and the `tasks` array is the single source of truth. Ask each pair to say which function runs when a checkbox is ticked (`toggleTask`, then `saveTasks`, then `renderTasks`).

Topic 8: Git and GitHub (slides 15–16, 15 min)

Teach: slide 15's picture: `main` stays production-ready, and each feature gets its own branch (C2 and C4 on the green line) that merges back as C5. Slide 16 is the four-step lifecycle. Ten commands, in the order students use them:

```
# 1. One-time setup (done before the day)
git config --global user.name 'Your Name'
git config --global user.email 'you@example.com'

# 2. Get the project: clone the pair's template repo
#    (git init, shown on slide 16, is only for a brand-new local project)
git clone https://github.com/<username>/<repo>.git
cd <repo>

# 3. See where you are
git status
git branch                      # * marks the current branch

# 4. Create and switch to a feature branch (slide 16, step 1)
git switch -c feature/add-task
```

```
# 5. Stage your changes (step 2)
git add .

# 6. Save a snapshot with a clear message
git commit -m 'Implement add logic'

# 7. Read the history (the C1, C2, C3 of slide 15)
git log --oneline

# 8. Push the branch to GitHub (step 3)
git push -u origin feature/add-task

# 9. Open a Pull Request on GitHub, review, merge (step 4), then refresh
local main
git switch main
git pull origin main

# 10. Resolve a merge conflict (details below)
git switch feature/daily-tip
git pull origin main             # Git stops: CONFLICT in script.js
# edit the file, then:
git add script.js
git commit -m 'Resolve merge conflict'
git push
```

The planned conflict. The starter's last line is `renderTasks();`. The persistence branch rewrites that one line as `tasks = loadTasks(); renderTasks();`. The daily-tip branch rewrites the same line as `renderTasks(); loadDailyTip();`. Whichever is merged second hits a conflict. Tell students to edit that existing line rather than adding new lines, otherwise Git merges quietly. Git shows:

```
<<<<<<< HEAD
tasks = loadTasks(); renderTasks();
=======
renderTasks(); loadDailyTip();
>>>>>>> feature/daily-tip
```

The fix is to keep both ideas and delete the three marker lines:

```
tasks = loadTasks();
renderTasks();
loadDailyTip();
```